Large-Scale Hospital Event Analytics

Project P35, Cloud Application and Development, UPES. Apratim Saxena.

What was needed

The project needed a system that takes in millions of hospital events (admissions, discharges, transfers, ICU moves) and turns them into answers an administrator can use: how many patients arrive, how full the beds are, and which days or hours were unusually busy or quiet. It had to stay fast as the data grows, stay correct when parts of it fail, and be clear to staff who are not technical.

Why I made it

This is project P35 for Cloud Application and Development. I chose to build it end to end instead of only describing it, because the topics of the course (handling large data, spreading work across many machines, surviving failures, securing access, and measuring performance) only become clear when a working system has to deal with them. A hospital is a good setting for this: events never stop, the data is large and time-stamped, and the people who need the answers ask practical questions, such as whether there are beds for tonight or why Tuesday went so badly, not technical ones.

I also wanted the result to be honest and checkable. The data is simulated, so no real patient information is involved, but it is generated with realistic weekly and daily rhythms and with busy periods that I hid on purpose, so I can test whether the system finds what I planted. Everything is measured, and the numbers are reported as they came out, including the less flattering ones, such as four workers giving a speed-up of 1.7 times rather than four times.

How it works: what a visitor sees first

A visitor who opens the demo link lands directly on the Overview page. On the full system they first sign in, and what they can do depends on their role: a viewer can read everything, an analyst can also run analyses, and an administrator can also manage access keys. On the left is a short menu of five pages (Overview, Admission patterns, Resource utilisation, Unusual workloads and Reports) with Settings below it, because starting analyses and connecting data sources are set-up tasks rather than daily ones. At the top of the page are the dates the results cover and two controls, a time period (last 30 days, 90 days, 12 months, or all time) and a department filter. Both stay selected as the visitor moves between pages.

Underneath is a row of key figures, each compared with the period before: patients admitted (147,663 over the last twelve months), average length of stay, beds in use stated as a count, for example 596 of 1,019 with 423 free, and the number of unusual workload periods. Below that are the admissions trend, the departments compared, bed use for each department with the busiest hour marked, and the most serious recent unusual periods. Everything is written in plain words, and counts come before percentages, because administrators manage beds, not ratios.

How it works: what happens inside

Take an example: an analyst asks for a full analysis of the ICU from 1 January to 31 March 2025. First, the analyst signs in and submits the request. The system checks the person's role and the dates, records the job as queued, and answers at once with a job number, so the page never waits for the work. A progress bar on the Analyses page then follows the job.

Second, the date range is split into small independent pieces, one for each day and each department. For the ICU over these 90 days that is 90 pieces; for all eight departments across three years it is 8,768. Admissions and bed counts add up cleanly across days and departments, so a piece needs no knowledge of any other piece.

Third, the pieces go onto a queue and a pool of worker processes takes them in parallel. Each worker reads only its own day of ICU events from the stored record, which is organised by month so the database touches only the relevant slice. It counts admissions, discharges and transfers, measures how long patients stayed, and works out how many beds were occupied each hour, producing a small set of partial totals. Because pieces share nothing, adding workers makes the whole job faster.

Fourth, the partial totals are grouped and merged. For each department the merged daily and hourly figures are compared with what is normal for that department on that weekday and hour, based on the previous five weeks. A period far above or far below normal is flagged and given a severity of moderate, high or critical. Flagged values are left out of later baselines, so one surge does not teach the system that surges are normal.

Fifth, the final figures are written to the results tables in one bulk step that replaces earlier values for the same days, so running an analysis twice, or repeating a piece after a failure, never creates duplicates. Each result row records which job produced it, and the raw events are never changed, which keeps the record auditable. The pages read these finished tables, so they load in a few milliseconds however many raw events exist. If a worker stops midway, its unfinished pieces are handed to the others after 45 seconds and the final numbers come out identical.

How it is working

The system runs as a set of containers (web service, four workers, database, queue and object store) and is covered by 90 automated tests. The demo data is 1,000,000 simulated events from 2023 to 2025 across eight departments. On one laptop, a full analysis of all of it takes about 45 seconds with one worker and 27 seconds with four.

I measured five things, each to answer a question. Processing time at 10,000, 100,000 and 1,000,000 events with one, two and four workers shows whether more workers really help; four workers gave 1.7 times the speed, not four times, because the final merging step cannot be split. Page response time as the data grew a hundred times shows whether pages stay fast for administrators; the median stayed between 4 and 9 milliseconds at every size. A load test with 500 and 1,000 simultaneous users shows where the system breaks; at 500 users it served 387 requests per second with a 95th percentile of 28 milliseconds and no errors, and at 1,000 the only refusals were deliberate rate limits. A failure test, stopping a worker at 40 percent of a one-million-event job, shows whether a crash can change the answer; the job finished with identical results after about 46 seconds of recovery. Finally, detection accuracy: all seven incidents I hid in the data were found, although the detector also reports some periods that match no planted incident, which I report rather than hide.

Problems I faced

The hardest part was deployment, and the reasons were practical. I planned to host the full system on a free Oracle Cloud server, but its signup needs a credit card for identity verification and I do not have one, so that route was closed. A temporary public link from my own laptop worked, but it exists only while the laptop, Docker and the connection stay on, which is not acceptable for an evaluation. Docker Desktop on my laptop also kept crashing at startup because of stale socket files left behind by an unclean shutdown, and I had to move those aside before the system would start. Preparing the hosted configuration was still worthwhile, because testing it found a real bug: a fresh build installed a newer version of a messaging library that needs a newer Redis client than the one I had fixed, so every worker crashed on start. It had gone unnoticed because my existing images were built earlier. I fixed it by pinning every package to the tested versions, so any fresh install builds the same system.

The final solution is a read-only snapshot hosted free on GitHub Pages. I recorded every response the interface needs from the running system by driving every page, period and department, and the published site answers from that recording instead of from a server. The numbers are real, but starting a new analysis, sending events and downloading CSV files only work on the full system, which starts with one command from the repository. The complete hosted configuration, with HTTPS, generated secrets and one setup script for a fresh server, is also in the repository for anyone who has a server.

Earlier in the build, three other problems taught me a lot. Under heavy load the system collapsed because database connections were all in use while requests waited on slow steps; fixing three causes took the 500-user test from heavy failures to none. A killed worker's tasks took about two minutes to be handed over because of the queue's default check interval, and changing it brought recovery down to 46 seconds. And building the interface exposed a flaw in my simulated data: six of the eight wards looked over capacity in a normal year, so I resized the wards from the simulated busiest hour.

How to check the demo

Open https://apratim-1407.github.io/hospital-event-analytics/ and try the following in order; no sign-in is needed. On Overview, change the period and the department and watch every figure update. On Resource utilisation, read the beds in use and beds free for each department, and the hours above 90 percent compared with the previous period. On Unusual workloads, click any row to see why it was flagged. On Reports, read the measured results and use the print button for a one-page summary. Under Settings, Analyses shows the history of analyses and Integrations shows how events are sent in. The demo is a read-only copy of real recorded data. The source code, the instructions to run everything yourself, and the longer documents are at https://github.com/apratim-1407/hospital-event-analytics.